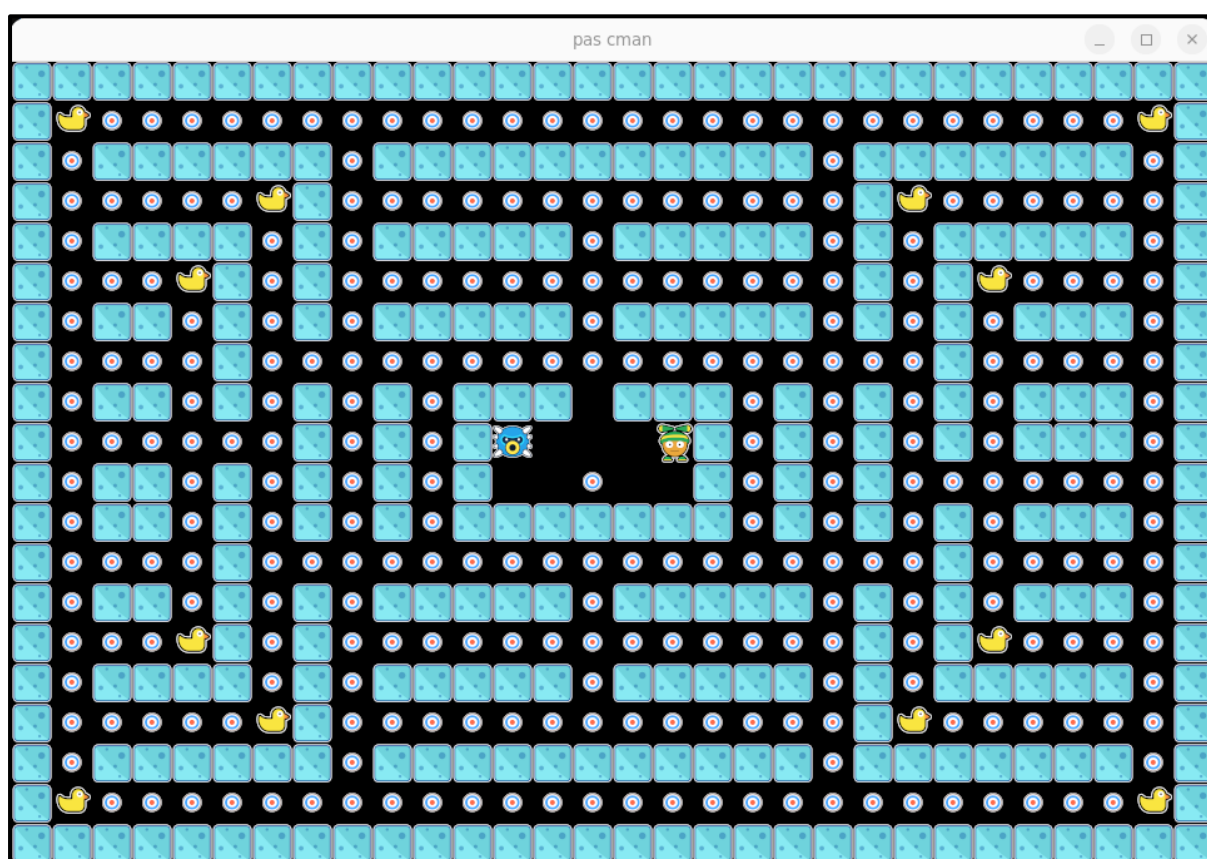


Projet de programmation système : Pas Cman

Xavier Gillard, Anthony Legrand, Jérôme Plumet

Introduction

Dans ce projet, vous aurez l'occasion d'approfondir et mettre en pratique vos connaissances nouvellement acquises en matière de programmation système en implémentant une variante multi-joueur du jeu 'Pas Cman'. Pas Cman est un jeu de plateforme qui rappelle les premiers jeux vidéo grand public au début des années 80.



Dans la version du jeu que vous allez implémenter, il y aura deux protagonistes (les 2 joueurs). Chaque joueur cherche à manger autant de nourriture que possible sur le tableau de jeu. Comme vous pouvez le voir dans l'image ci-dessus, il existe 2 sortes de nourritures sur le plateau : la nourriture normale (les petites cocardes) et les superfood (les canards de bain). Chaque superfood mangée donne autant de points qu'avaler 17 pastilles de nourriture normale.

La configuration initiale du plateau (càd. la position des murs, de la nourriture et des joueurs) est définie dans un fichier texte appelé "map". Nous vous fournirons trois maps (la capture ci-dessus correspond au fichier "map1.txt"). Toutes les maps ont les mêmes dimensions.

Dans cette version du jeu, les règles sont très simples :

- Initialement, les deux joueurs apparaissent à une position qui est prédéfinie par la carte.
- La partie commence dès que deux joueurs sont connectés (le plateau de jeu apparaît).
- Les deux joueurs ont le droit de se déplacer sur la carte autant qu'ils le souhaitent.
- Il n'y a pas de notion de tour, chacun peut décider de bouger (ou pas) comme il l'entend. La seule contrainte étant que lorsqu'un joueur essaie de faire un mouvement qui lui ferait traverser un mur, il reste bloqué et son personnage ne se déplace pas.
- Lorsqu'un joueur se déplace sur une case comprenant de la nourriture, il mange la nourriture présente sur cette case.
- Si deux joueurs arrivent sur la même case en même temps, la partie s'arrête.
- S'il n'y a plus de nourriture sur le plateau, la partie s'arrête.
- Le joueur dont le personnage a mangé le plus de nourriture au moment où la partie s'arrête est le gagnant.

Indications de programmation

Le projet à réaliser est une application client-serveur qui utilise des sockets TCP/IP pour implémenter le dialogue entre le gestionnaire du jeu (le serveur) et les joueurs (les clients) ainsi qu'une mémoire commune pour partager l'état du jeu (càd. la carte du jeu, l'emplacement de la nourriture et des superfood, les score et la position des joueurs).

Un joueur (un client) demande l'autorisation de jouer au gestionnaire du jeu (le serveur). Pour ce faire, basez-vous sur la solution de l'exercice 6.1. Si la partie n'a pas encore démarré et si le nombre maximum de joueurs n'est pas atteint, le joueur est accepté, sinon il est refusé. Le serveur envoie sa réponse au joueur. Avant de démarrer une partie, le serveur attend qu'il y ait deux joueurs qui aient été acceptés. S'il n'y a qu'un seul joueur enregistré au bout d'une période de 30 secondes, il est déconnecté et le serveur démarre la phase d'inscription pour une nouvelle partie¹.

De son côté, le client devra communiquer avec un processus d'interface graphique (**`pas-cman-ipl`** fournie) ET avec le serveur afin de permettre au joueur humain d'interagir efficacement avec le jeu vidéo.

Concrètement, dans ce projet, les étudiants seront chargés de concevoir et de programmer différentes parties du jeu multijoueur. A cet effet, ils programmeront **trois exécutables** : **`pas\_server`**, **`pas\_client`** et **`pas\_labo`**.

¹ Cette limitation de la période d'inscription sert à éviter le scénario où un premier joueur s'inscrit mais part se faire un café parce qu'il attend trop longtemps un second joueur ; dans ce cas, le risque est que, lorsqu'un second joueur finit par s'inscrire, celui-ci doive attendre trop longtemps le retour du premier joueur.

1. Le programme `pas_server`

Le programme ``pas_server`` est le point d'entrée qui permet à plusieurs joueurs de jouer une partie de Pas Cman ensemble. Il s'agit d'un serveur qui implémente le paradigme BLAB (**B**ind - **L**isten - **A**cept - **B**egin) pour écouter sur un port TCP.

Pour chaque connexion entrante, ``pas_server`` va créer un nouveau processus qui sera responsable d'un joueur en particulier, appelons-le ``client_handler``. En plus de son rôle de pur serveur, ``pas_server`` sera aussi responsable de charger la map du jeu et de créer (et détruire) les ressources partagées nécessaires pour maintenir un état du jeu qui soit cohérent pour tous les ``client_handler``. L'accès à la mémoire partagée devra être sécurisé par l'utilisation de sémaphores. Le serveur est responsable à la fois de la création et de la destruction des IPC.

Concernant l'implémentation des sockets, le numéro du port sur lequel on peut se connecter au serveur est à préciser comme argument de chacun des deux programmes, ``pas_server`` et ``pas_client``.

Le serveur ne s'éteint jamais. Cependant, dans le cadre de ce projet, il doit être possible de l'arrêter en lui envoyant un signal SIGINT (Ctrl-C). Notez que le serveur ne peut pas interrompre la partie en cours. Cela signifie que la phase d'inscription de 30 secondes et la phase de jeu ne seront pas interrompues par Ctrl-C. A la réception d'un SIGINT, le serveur s'arrêtera après la fin de la partie courante.

1.1. Les sous-processus (`client_handler`)

Chacun des sous-processus créés par le programme ``pas_server`` devra gérer les interactions avec le client d'un des joueurs. Chacun de ces sous-processus écoutera donc les demandes de mouvement émises par le client dont il a la charge, puis il essaiera de modifier l'état partagé du jeu en respectant les règles du jeu².

2. Le programme `pas_client`

Le programme ``pas_client`` est le programme qui est utilisé par un joueur lorsqu'il veut participer à une partie de Pas Cman.

Ce programme aura donc deux rôles : d'une part, il devra créer un sous-processus dans lequel il exécutera le programme ``pas-cman-ipl`` (qui vous est fourni, vous ne devez pas le programmer vous-même). Ce programme ``pas-cman-ipl`` implémente la gestion du clavier et toute la partie graphique mais il ne connaît absolument pas la logique du jeu.

² En particulier, il n'effectuera aucun coup illégal (si un joueur veut aller dans un mur par exemple).

D'autre part, le programme ``pas_client`` écoutera sur la sortie standard de ``pas-cman-ipl`` pour connaître les mouvements que le joueur désire faire. Il transmettra ensuite ces ordres au sous-processus du serveur (`client_handler`) avec lequel il communique. Par ailleurs, l'interface graphique ``pas-cman-ipl`` devra être tenue au courant (en les lisant sur son entrée standard) de tous les événements du jeu qui se sont produits (envoyés par le `client_handler` du joueur).

3. Le programme `pas_lab0`

Le programme ``pas_lab0`` lancera un ``pas_server`` et deux joueurs ``pas_client`` sur la même machine (localhost) afin d'installer un petit laboratoire de test qui puisse servir à valider les implémentations du serveur et du client. Le labo permettra ensuite de jouer une partie de Pas Cman pré-enregistrée afin de pouvoir valider le comportement des différents programmes. Nous vous fournirons quelques scénarios d'exemples en annexe à ce projet.

Lorsque le programme ``pas_lab0`` se termine (fin du scénario), il doit envoyer un signal à tous les clients et serveurs qui ont été lancés afin de s'assurer qu'eux aussi se terminent.

4. Gestion des arrêts et situations exceptionnelles

De manière générale, ni le serveur, ni les clients ne doivent gérer les arrêts brutaux (autres que SIGINT). En particulier, on fait l'hypothèse que personne n'effectuera jamais un ``kill -9`` pour tuer un de vos programmes. Vous ne devez donc pas gérer ces situations d'arrêts brutaux.

5. Utilisation des moyens de communication

Les clients et le serveur ne tournent pas nécessairement sur la même machine. La mémoire partagée n'est accessible que par des processus qui tournent sur la même machine que le serveur. Pour ce faire, le serveur doit créer un fils par client qui joue le rôle de client local (`client_handler`) et accède à la mémoire partagée. Les clients distants communiquent alors avec le ``client_handler`` qui leur est attribué (via sockets) et effectuent les affichages et autres entrées/sorties à destination du joueur. Les informations qui circulent entre un joueur et le serveur transitent par des sockets TCP et les informations connues de tous sont conservés dans une mémoire partagée.

Les sémaphores permettront aux clients locaux (`client_handlers`) de se mettre en attente afin d'éviter que l'état partagé ne soit modifié par les deux joueurs en même temps, ce qui pourrait le rendre inconsistant.

6. Contraintes pratiques

Pour mener à bien ce projet, il est indispensable de travailler de manière modulaire. Il vous est demandé de respecter la découpe en modules suivante :

- réseau (tcp-ip)
- jeu (logique assurée par les client_handlers)
- serveur (*main*)
- client (*main*)

Pour compiler et faire l'édition de liens des programmes, un « makefile » doit être écrit.

Quand l'application se termine, aucun IPC ne peut traîner sur le système!!!

Tous les appels systèmes doivent être testés. En cas d'erreur, vous pouvez simplement fermer l'application.

Nous vous conseillons d'utiliser le module *utils* qui vous est fourni pour cette UE.

7. Indications utiles

Toutes les ressources nécessaires à l'implémentation de votre jeu³ seront rendues disponibles sur mooVin à la seconde semaine du projet (voir section Planning).

Commencez par bien lire les informations qui vous sont données dans le répertoire "**student_kit**". En particulier, lisez bien le contenu du fichier **README** et du header "**pascman.h**".

Assurez-vous de bien comprendre le code d'exemple qui vous est fourni dans le répertoire avant de vous atteler à l'implémentation des programmes ``pas_client`` et ``pas_server``.

Nous vous conseillons de travailler localement sur vos propres PC (dans un environnement Linux/Unix ou MacOS). Cela permettra de pallier les insuffisances du serveur courslinux et à tous les groupes de travailler en même temps sans interférer les uns avec les autres. Dans le cas où vous travaillez sur les pc de l'école, utilisez Ubuntu sous WSL (adressez-vous aux enseignants pour plus d'info). Dans tous les cas, nous vous conseillons d'utiliser Git pour vous permettre de gérer votre code en vous répartissant le travail au sein de votre groupe.

³ Si le code de l'interface graphique vous intéresse, ou si vous décidez de recompiler l'interface graphique pour votre propre machine, vous pourrez en trouver le code sur le repository git du projet (<https://github.com/xgillard/pas-cman-ipl>). Notez que cette interface graphique est programmée en Rust (un autre langage de programmation système compatible avec le C).

8. Délivrables

8.1. L'analyse

Nous vous demandons de réaliser l'analyse de l'application.

Avant de se lancer dans la programmation, il est en effet prudent de commencer par analyser le problème posé. Il vous est demandé de remplir le document « **analyse_application.docx** », disponible sur MooVin. Les deux séances de la première semaine du projet y seront consacrées (voir Planning ci-dessous).

8.2. Le code final

Nous vous demandons de remettre sur MooVin l'ensemble de votre code source (documenté!), ainsi qu'un Makefile permettant la compilation des différents programmes exécutables au terme des 3 semaines du projet (voir Planning ci-dessous).

Nous vous rappelons que la politique de la Haute École en matière de **plagiat** est très stricte. Nous vous signalons qu'un système automatique de détection de plagiat sera appliqué à votre code. Attention que les IAs ont tendance à générer toujours le même code...

8.3. Défense orale

Vos projets seront testés en votre présence le **jeudi 15 mai**. Le code qui sera exécuté sera celui soumis sur MooVin et pas un autre ! Les scénarios de test vous seront communiqués durant le projet.

9. Planning

Date	Action
Ven. 11 avril	• Publication énoncé & démo en vidéo
Semaine 1	• TODO : Choix des groupes (si des groupes de 4 étudiants ne sont pas constitués lors de la 1^e séance, les enseignants les définiront) • TODO : Analyse du projet
Ven. 18 avril - 14h	• TODO : Soumission du document d'analyse • Publication de l'architecture à implémenter & des ressources « student_kit »
Semaine 2	• TODO : Implémenter le jeu en respectant l'architecture fournie
Semaine 3	• TODO : Implémenter le jeu (suite et fin)
Ven. 9 mai - 20h	• TODO : Soumission du code source de vos programmes
Jeu. 15 mai	• Défense orale du projet (un horaire précis + scénarios de tests vous seront communiqués en temps voulu)

10. Evaluation

La note du projet est répartie de la sorte :

- **20%** : validation de l'analyse [note de groupe]
- **30%** : présence active aux 5 séances du projet [note individuelle]
- **50%** : code et défense du projet (contrôle du code + exécution des tests lors de la défense) [note de groupe]